

networking 101

alexander

August 17, 2026

The TCP/IP model is a useful abstraction that helps us reason about data as it moves around in a computer network. The model gives us a way to reason about how data is handled by different protocols at different stages of communication. The TCP/IP model consists of four layers: Application, Transport, Internet, and Network Access. Data can be passed between layers as it is processed, moving down the stack when being prepared for transmission and up the stack when being processed by the receiving device. The units of data handled at each layer are called protocol data units (PDUs), with names such as a segment, packet, and frame depending on the layer.

IPv4 uses a 32-bit address, providing 2^{32} , or 4,294,967,296, possible address values, although not all of this space is available for ordinary globally routable host addresses. IPv4 addresses were historically divided into Classes A, B, C, D, and E. Classes A, B, and C were used for different sizes of unicast networks under the original classful addressing system, while Class D (224.0.0.0–239.255.255.255) is used for multicast and Class E (240.0.0.0–255.255.255.255) was reserved for experimental or future use. Classful addressing eventually proved inefficient because the fixed network sizes did not correspond well to the actual requirements of organizations, leading to wasted address space and inefficient allocation. CIDR replaced this system with variable-length prefixes, allowing the boundary between the network and host portions of an address to be chosen according to the required network size. Within a conventional IPv4 subnet, the address with all host bits set to zero identifies the network itself, while the address with all host bits set to one represents the directed broadcast address, leaving $2^n - 2$ traditionally usable host addresses when n bits are available for hosts. CIDR made IPv4 allocation substantially more efficient, but it could not overcome the fundamental limitation of a 32-bit address space. As the number of Internet-connected devices increased, the conservation and reuse of IPv4 address space became increasingly important. In 1996, the Internet Engineering Task Force (IETF) formally reserved three IPv4 address blocks for private networks in RFC 1918: 10.0.0.0/8, 172.16.0.0/12, and 192.168.0.0/16. These addresses are intended for use within private networks and are not globally routable across the public Internet. Because they do not need to be globally unique, the same private address ranges can be reused independently by different organizations and networks. This allowed enormous numbers of internal hosts to operate without each host requiring its own globally routable IPv4 address. Network Address Translation (NAT) then provided a mechanism for these privately addressed hosts to communicate with the public Internet by translating private addresses into globally routable public addresses at the net-

work boundary. Static NAT provides a persistent one-to-one mapping between an internal and external address, while dynamic NAT can allocate translations from a pool of public addresses. The most common form for allowing many internal hosts to share a small number of public IPv4 addresses is Port Address Translation (PAT), often called NAT overload. PAT uses transport-layer port numbers in addition to IP addresses so that multiple simultaneous connections from different internal hosts can share the same public address while maintaining distinct translation entries. A NAT device therefore maintains state describing these mappings and uses the resulting translation information to associate returning traffic with the appropriate internal host and connection. NAT consequently became a practical mechanism for conserving IPv4 address space, although it does not increase the actual size of the IPv4 address space and is not inherently a security mechanism; its primary purpose is address translation, while firewall and filtering policies provide traffic-control and security functions.

The Transport layer provides communication between applications running on different hosts, operating above the Internet layer and using protocols such as TCP and UDP to determine how application data is transported between endpoints. While IP is responsible for delivering packets between hosts, it does not by itself identify which application or process should receive that data; the Transport layer uses port numbers to provide this distinction, allowing a host to simultaneously communicate with many different applications and services. TCP and UDP take fundamentally different approaches to this task. UDP provides a minimal, connectionless transport service in which application data is placed into datagrams and delivered without establishing a connection or providing built-in guarantees of delivery, ordering, retransmission, flow control, or congestion control. TCP provides a substantially more sophisticated transport service by establishing a connection and maintaining state about the communication between the two endpoints. TCP uses sequence numbers and acknowledgments to track data, retransmits data that is determined to have been lost, preserves the ordering of the byte stream presented to the application, and uses flow-control and congestion-control mechanisms to regulate how much data can be transmitted and how quickly it should be sent. Consequently, the Transport layer is not simply concerned with moving data between two IP addresses; it provides the mechanisms by which individual application processes can communicate across the network and, depending on the transport protocol selected, can provide additional guarantees and control over how that communication occurs. At the operating-system level, this communication ultimately interacts with processes through mechanisms such as sockets, with the operating system maintaining transport-layer state, associating ports with applications, buffering received and transmitted data, and implementing much of the TCP or UDP behavior on behalf of applications. This makes the Transport layer an important boundary between networking and the operating system: applications generally do not construct TCP segments or manipulate Ethernet frames themselves, but instead use operating-system networking interfaces to request communication, leaving the protocol stack to construct, transmit, receive, and process the underlying protocol data units.

The Transport layer provides communication between applications running on different hosts, operating above the Internet layer and using protocols such as TCP and UDP to

determine how application data is transported between endpoints. While IP is responsible for delivering packets between hosts, it does not by itself identify which application or process should receive that data; the Transport layer uses port numbers to provide this distinction, allowing a host to simultaneously communicate with many different applications and services. TCP and UDP take fundamentally different approaches to this task. UDP provides a minimal, connectionless transport service in which application data is placed into datagrams and delivered without establishing a connection or providing built-in guarantees of delivery, ordering, retransmission, flow control, or congestion control. TCP provides a substantially more sophisticated transport service by establishing a connection and maintaining state about the communication between the two endpoints. TCP uses sequence numbers and acknowledgments to track data, retransmits data that is determined to have been lost, preserves the ordering of the byte stream presented to the application, and uses flow-control and congestion-control mechanisms to regulate how much data can be transmitted and how quickly it should be sent. Consequently, the Transport layer is not simply concerned with moving data between two IP addresses; it provides the mechanisms by which individual application processes can communicate across the network and, depending on the transport protocol selected, can provide additional guarantees and control over how that communication occurs. At the operating-system level, this communication ultimately interacts with processes through mechanisms such as sockets, with the operating system maintaining transport-layer state, associating ports with applications, buffering received and transmitted data, and implementing much of the TCP or UDP behavior on behalf of applications. This makes the Transport layer an important boundary between networking and the operating system: applications generally do not construct TCP segments or manipulate Ethernet frames themselves, but instead use operating-system networking interfaces to request communication, leaving the protocol stack to construct, transmit, receive, and process the underlying protocol data units.

The Transport layer provides communication between applications running on different hosts, operating above the Internet layer and using protocols such as TCP and UDP to determine how application data is transported between endpoints. While IP is responsible for delivering packets between hosts, it does not by itself identify which application or process should receive that data; the Transport layer uses port numbers to provide this distinction, allowing a host to simultaneously communicate with many different applications and services. TCP and UDP take fundamentally different approaches to this task. UDP provides a minimal, connectionless transport service in which application data is placed into datagrams and delivered without establishing a connection or providing built-in guarantees of delivery, ordering, retransmission, flow control, or congestion control. TCP provides a substantially more sophisticated transport service by establishing a connection and maintaining state about the communication between the two endpoints. TCP uses sequence numbers and acknowledgments to track data, retransmits data that is determined to have been lost, preserves the ordering of the byte stream presented to the application, and uses flow-control and congestion-control mechanisms to regulate how much data can be transmitted and how quickly it should be sent. Consequently, the Transport layer is not simply concerned with moving data between two IP addresses; it provides the mechanisms by which individual application processes can communicate across the network and, depending on

the transport protocol selected, can provide additional guarantees and control over how that communication occurs. At the operating-system level, this communication ultimately interacts with processes through mechanisms such as sockets, with the operating system maintaining transport-layer state, associating ports with applications, buffering received and transmitted data, and implementing much of the TCP or UDP behavior on behalf of applications. This makes the Transport layer an important boundary between networking and the operating system: applications generally do not construct TCP segments or manipulate Ethernet frames themselves, but instead use operating-system networking interfaces to request communication, leaving the protocol stack to construct, transmit, receive, and process the underlying protocol data units.

At the operating-system level, this interaction is commonly exposed to applications through sockets. A socket provides an interface through which a process can communicate using a transport protocol, with the operating system maintaining the state and resources associated with that communication. Port numbers allow the operating system to associate incoming traffic with the appropriate socket and therefore with the appropriate process. A TCP connection can be identified by a combination of the source IP address, source port, destination IP address, and destination port, commonly called the four-tuple. This allows a host to maintain many simultaneous connections, even when multiple connections use the same destination port, because the complete combination of addresses and ports distinguishes one connection from another. Client applications commonly use dynamically assigned ephemeral ports when initiating connections, while servers generally listen for incoming connections on known ports associated with particular services.

For TCP, establishing a connection requires the endpoints to first establish the state necessary for communication. This occurs through the three-way handshake, in which the endpoints exchange SYN and ACK segments and establish their initial sequence-number information. Once the connection is established, TCP presents the application with an ordered byte stream rather than treating each transmitted segment as an independent piece of application data. TCP uses sequence numbers to identify positions within this stream and acknowledgments to indicate which data has been received. If data is lost, TCP can detect the loss and retransmit the missing data, allowing the application to receive an ordered and reliable stream even though the underlying IP network itself provides no guarantee that packets will arrive, arrive only once, or arrive in order.

TCP must also regulate communication in two different ways. Flow control prevents a sender from overwhelming the receiving host by limiting how much unacknowledged data can be in flight based on the receiver's available buffer space. Congestion control, in contrast, concerns the state of the network itself and attempts to prevent a sender from contributing to network congestion by adjusting the rate at which data is transmitted. These mechanisms, along with retransmission, sequencing, acknowledgments, timers, and connection state, are part of what makes TCP substantially more complex than UDP. Much of this behavior is implemented by the operating system's networking stack rather than by the application itself, providing an example of how the Transport layer sits at the boundary between application processes and the underlying network.